
Software Testing Process

Information Management 2026

Group: Software Testing Process 2

Member names: Torben Freise, Conrad Clough, Anton Ruban

Emails: t.a.freise@student.vu.nl, c.j.clough@student.vu.nl, a.r.ruban@vu.nl

VUnetIDs: tfr216, cch250, rcv789

June 1, 2026

Contents

1	Introduction	2
2	Process Description	3
2.1	Actors	3
2.2	Activities	3
3	Transition Systems	5
3.1	Overall System	5
3.1.1	State-transition Diagram	6
3.1.1.1	Description	6
3.2	Bug Fix	7
3.2.1	State-transition Diagram	9
3.2.1.1	Description	10
4	Petri Nets	11
4.1	Overall Software System Petri Net	11
4.1.1	Petri Net Graph	13
4.1.1.1	Description	14
4.1.2	Reachability Graph	14
4.1.3	Property Analysis	14
4.2	Bug Fix Petri Net	16
4.2.1	Petri Net Graph	18
4.2.1.1	Description	18
4.2.2	Reachability Graph	20
4.2.3	Property Analysis	21
5	Reflection	22

Introduction

Software testing is a process within software development that's aimed at ensuring that software meets specified requirements and functions reliably. It involves several actors, such as developers, testers, and stakeholders who work together to verify the correctness of the software using unit, integration, system, and acceptance testing. These tests aim to detect logic errors and ensure correct behaviour so that the software meets user expectations.

In chapter 2, the software testing process will be described in more detail, including its phases and the roles of the involved actors. Two transition systems are presented in chapter 3 to depict the different states and the possible transitions between these states. Additionally, chapter 4 contains two Petri nets corresponding to the transition systems provided in the previous section with their respective reachability graphs and property analysis. Lastly, chapter 5 contains a reflection discussing the advantages and disadvantages of the modelled process, and takeaways of the project and the work division.

Process Description

2.1 Actors

There are several actors involved in the software testing process. These actors and their respective roles are as follows:

Developers. Developers are responsible for implementing features and bug fixes, plus writing and performing tests, such as unit tests. Their role is to ensure that the individual components of the software system function as expected before integration. Developers creating bug fixes are responsible for creating the fix and (re)submitting it until accepted.

Testers. Testers design, execute, and evaluate further tests such as integration and system tests. They're responsible for identifying issues and ensuring the system behaves as expected in a broad sense with a focus on user expectations. When a bug fix is submitted by a developer and passes automated tests, a tester will then manually review and test it. Importantly, testers are not a separate group of actors that exist at all times - developers become testers when required and revert to developers when tests have been created or a bug fix is complete.

DevOps Engineers. DevOps engineers support testing by creating the environments and pipelines necessary for CI/CD. In particular, they ensure previously created tests are ran automatically, alongside security scans and deployment.

Stakeholders. Stakeholders such as clients define the requirements for the software system, and periodically review the software to ensure that it meets their needs and aligns with their idea. The requirements come from what users need. They're primarily involved in acceptance testing but are communicated with throughout all stages of the process. They begin the development pipeline by requesting a feature.

End Users. End users are the target audience of the system and the ones that the stakeholders want to cater to. To this end, they can be involved in acceptance testing to give feedback on the user experience of the software. They also report bugs.

2.2 Activities

The process of testing a software system can be split into the following steps:

Development. Developers implement the system based on specific requirements they receive from stakeholders, or revise existing code to fix a bug or pass a failing test. They write their code in a way that allows for it to be modular and easily tested.

CI/CD Pipeline Setup. DevOps engineers are responsible for setting up a CI/CD pipeline early, so that developers and testers can have the software be automatically evaluated and standardised with minimal friction. This involves the automation of test execution, code formatting, and static analysis, and security scanning, normally done using pre-commit hooks for the software's Git repository.

Unit Testing. Developers write unit tests that verify that their modules function as expected when isolated. This is closely tied with development and often done alongside it.

Integration Testing. After modules are developed, testers create integration tests that ensure that the modules work together when combined and interacted with in a realistic way. This step identifies broad structural issues with the software.

System Testing. After integration testing, testers evaluate the complete system to verify it meets requirements. This is done in an environment that resembles the real world, and can identify performance or usability issues.

Acceptance Testing. Stakeholders and end users interact with the software in order to ensure that it satisfies requirements and expectations. This determines if the software is ready to be released or if further development needs to be done.

Bug Triage. When a bug is reported, a developer assesses how urgent it is and assigns a priority. Critical bugs are immediately assigned to a developer to be fixed and lower priority bugs are placed in a backlog, to be fixed when developers are available.

Bug Fixing. If any issues are found during testing, then developers address them and their fix is passed through the CI/CD pipeline. A failure at any point means the developer must continue to develop the fix until it passes. The fix is then also manually reviewed, and if it succeeds it's deployed.

Regression Testing. After bug fixes or updates, testers perform regression tests by rerunning previously used tests and ensuring that they all still pass and that existing functionality hasn't been affected.

Release. If the system passes acceptance testing, and has therefore met all the requirements from the stakeholders and is an acceptable experience for end users, then it's released.

Maintenance. After release, the feature enters maintenance mode where it's kept up to date by developers and testers. This is done to prevent issues such as newly discovered security exploits from affecting the software during its lifetime. Bug reports from users or developers trigger a bug fix. Additional features that may affect existing behaviour trigger a retest.

Transition Systems

This section includes two versions of transition systems for our software system process as described in chapter 2. The two different versions cover both the overall system and a focused view on a bug fix.

3.1 Overall System

Our first transition system comprises a triple (S, TR, s_0) :

- The finite state space

$$S = \{A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U\}$$

where

<i>A = Development</i>	<i>B = Local Testing</i>
<i>C = Attempt commit & run pre-commit hooks</i>	<i>D = Changes Committed</i>
<i>E = Changes Pushed</i>	<i>F = CI/CD Pipeline Start</i>
<i>G = Unit testing</i>	<i>H = Integration testing</i>
<i>I = System testing</i>	<i>J = CI/CD Pipeline Complete</i>
<i>K = Deployed to Staging Environment</i>	<i>L = Acceptance testing on Staging</i>
<i>M = Staging Deployment rolled back</i>	<i>N = Feature Abandoned</i>
<i>O = Merged to Main</i>	<i>P = Tagged / Released</i>
<i>Q = Deployed to Production</i>	<i>R = Deployment monitoring</i>
<i>S = Production Deployment rolled back</i>	<i>T = Changes reverted</i>
<i>U = Maintenance</i>	

- The transition relation

$$TR \subseteq S \times S = \left\{ \begin{array}{l} (A, B), (A, C), (B, A), \\ (C, D), (C, A), (D, E), \\ (D, A), (E, F), (F, G), \\ (G, H), (G, A), (H, I), \\ (H, A), (I, J), (I, A), \\ (J, K), (K, L), (L, O), \\ (L, M), (M, A), (M, N), \\ (O, P), (P, Q), (Q, R), \\ (R, U), (R, S), (S, T), \\ (T, A), (T, N) \end{array} \right\}$$

- The initial state $s_0 = A$

3.1.1 State-transition Diagram

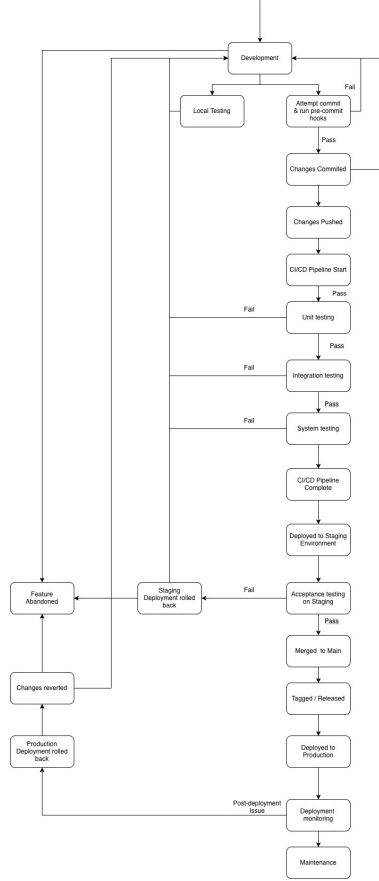


Figure 3.1: Transition diagram of full software system, start to finish for one feature

3.1.1.1 Description

This transition system models the full software development and delivery pipeline for one feature, from initial development through to live deployment and maintenance.

The initial state, *Development*, is where the feature is written by a developer. This feature is requested by a stakeholder for a user. A developer will continuously test a feature locally while working on it, modeled by the loop to *Local Testing*. From *Development*, the state can move to the terminal state *Feature Abandoned*, or move forward to *Attempt commit & run pre-commit hooks*. In this state, pre-commit hooks may run some static analysis on the changes as a basic test. If the hooks passed, the state moves to *Changes Committed*. From here, the developer can decide to push the changes, reaching *Changes Pushed*, or continue developing and thus batch their commits. Once the changes are pushed, the pipeline begins at *CI/CD Pipeline Start*. The pipeline performs automated *Unit testing*, *Integration testing*, and *System testing* on the code. If any tests fail, the feature goes back to the *Development* state for the developer to fix. In this diagram, the pipeline

tests are modeled as linear and *fail-fast*, where each test can only be run after the previous one passes. These tests may run in parallel in practice, but the crucial part is that they all must pass for the pipeline to succeed overall, reaching *CI/CD Pipeline Complete*.

The success of the pipeline is a pre-condition for merging the changes. Before merging, the changes should be deployed to a staging environment (*Deployed to Staging Environment*). This environment aims to replicate the production environment as closely as possible. *Acceptance testing* is usually done on staging in order to test not only the system but also the infrastructure. If it fails, the deployment enters *Staging Deployment rolled back*: the changes are reverted, and an older version is deployed to staging before development continues.

If no issues are found on staging, the feature is merged to main (*Merged to Main*) and a tag / release is created (*Tagged / Released*). After this, it can be deployed to production (*Deployed to Production*) and monitored (*Deployment monitoring*). If issues surface soon after deployment, the deployment may be rolled back (*Production Deployment rolled back*) and the changes reverted from main (*Changes reverted*), after which development can continue or the feature may be abandoned (*Feature Abandoned*).

If deployment goes well, the feature enters the *Maintenance* state. This state is a sink, as the merge request involving these changes will no longer be rolled back. Any issues found in relation to this feature, or improvement requests, will be treated as bug fixes or features in their own right.

3.2 Bug Fix

Our second transition system comprises a triple (S, TR, s_0) with initial state $s_0 = A$:

- The finite state space

$$S = \{A, B, C, D, E, F, G, H, I, J, K, L, M, N, O\}$$

where

$A = \text{Bug Reported}$	$B = \text{Triage begins}$
$C = \text{Priority assessment}$	$D = \text{Normal queue}$
$E = \text{Critical queue}$	$F = \text{Assigned to developer}$
$G = \text{In progress}$	$H = \text{Fix submitted}$
$I = \text{CI pipeline runs}$	$J = \text{Run automated tests}$
$K = \text{Security scan}$	$L = \text{In review}$
$M = \text{manual system testing}$	$N = \text{Bug Fix Accepted}$
$O = \text{Deployed to production}$	

- The transition relation

$$TR \subseteq S \times S = \left\{ \begin{array}{l} (A, B), (B, C), (C, D), (C, E), (D, F), \\ (E, F), (F, G), (G, H), (H, I), (I, J), \\ (J, K), (J, G), (K, L), (K, G), (L, M), \\ (L, G), (M, N), (M, G), (N, O) \end{array} \right\}$$

3.2.1 State-transition Diagram

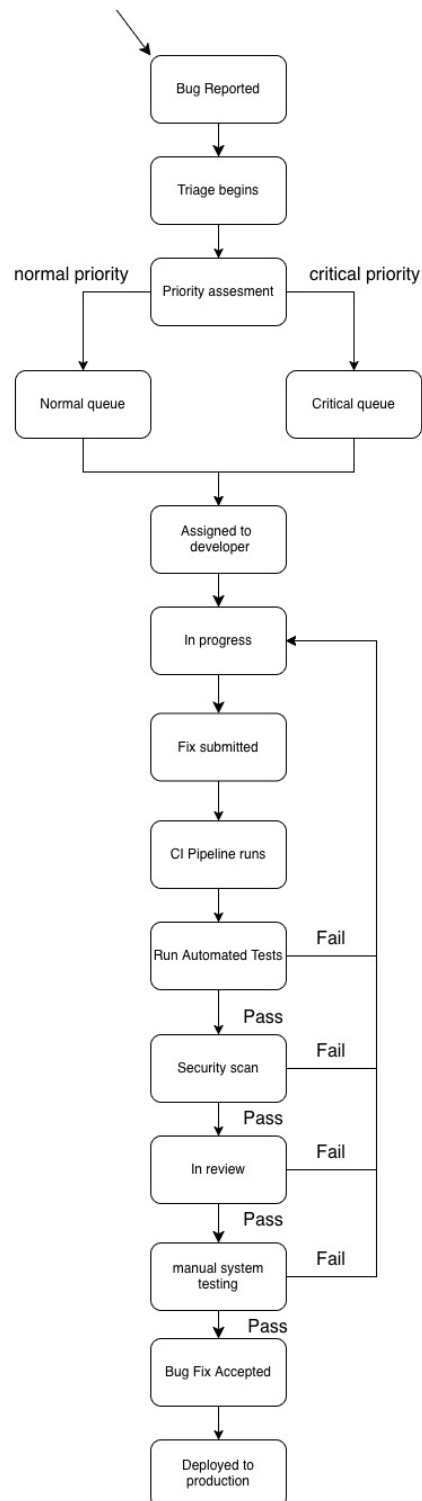


Figure 3.2: Transition diagram of a bug report, from report to deployment

3.2.1.1 Description

This transition system details the process of fixing a bug in the software. The initial state is the bug being reported; users and other developers must explicitly report bugs to begin the process. This allows for proper logging and tracking of the bug.

Afterwards, the bug is triaged. This involves assessing the impact of the bug, activities such as sorting it based on urgency, adding reproduction notes, and detailing who reported it and how. Once there is enough information, the bug is assigned a priority. For the sake of the model, we limit this to "normal" or "critical" priority. Based on the assigned priority, the fix is placed in a queue to wait until a developer is available to pick it up. As this cannot be effectively modeled in a state transition diagram, the rule that critical bugs will be dequeued before normal ones is implicit here.

After the bug is assigned to a developer, we enter the *In progress* state. This state is the entry-point to an area of our transition system containing multiple loops, as this is where fixes are repeatedly attempted and either accepted or rejected.

When a fix is submitted, the automated CI pipeline is run, which uses previously written unit, integration, and regression tests to evaluate whether the code contains trivial logical errors, introduces new bugs, or breaks existing behavior in the software. This is automated to reduce pressure on testers; rejecting a fix early with known-good tests with zero human intervention means that the bug fixing process requires only the assigned developer for as long as possible. A failure here takes us back to *In progress*, and the developer must change their fix and resubmit it.

After passing the CI pipeline run, a security scan is performed; failure also returns us to *In progress*. A pass means that the bug fix is then reviewed manually by another developer, acting as a tester. This stage is a less formal, more cursory process where the code is typically only read over glaring issues. If this review passes, then more thorough manual system testing is done in which the reviewing tester will interact with the software system with the bug fix applied. If this passes, the bug fix is accepted. Failure results in the bug fix bouncing back to the *In progress* state.

Finally, a fix that passes all tests is accepted and deployed to production.

Petri Nets

This section includes our two distinct Petri net systems, one for the overall software system and one for an individual bug fix, as well as their respective diagrams, reachability graphs, and analyses of process properties.

4.1 Overall Software System Petri Net

Our Petri net system comprises the quadruple (P, T, F, m_0) :

- Finite set of places

$$P = \{p_1, p_2, \dots, p_{26}\}$$

where

$p_0 = \textit{Development Start}$	$p_1 = \textit{Development Complete}$
$p_2 = \textit{Local Testing}$	$p_3 = \textit{Commit \& run pre-commit hooks done}$
$p_4 = \textit{Changes Committed}$	$p_5 = \textit{Changes Pushed}$
$p_6 = \textit{CI/CD Pipeline}$	$p_7 = \textit{Unit Testing}$
$p_8 = \textit{Integration Testing}$	$p_9 = \textit{System Testing}$
$p_{10} = \textit{CI/CD Pipeline Complete}$	$p_{11} = \textit{Deployed to Staging Environment}$
$p_{12} = \textit{Acceptance Testing on staging}$	$p_{13} = \textit{Roll Back Staging Deployment}$
$p_{15} = \textit{Abandon Feature}$	
$p_{17} = \textit{Merge to Main}$	
$p_{18} = \textit{Merge to main Complete}$	$p_{19} = \textit{Tagged and Released}$
$p_{20} = \textit{Deployed to Production done}$	$p_{21} = \textit{Deployment Monitoring}$
$p_{22} = \textit{Post Deployment Issue}$	$p_{23} = \textit{Production Rolled Back}$
$p_{24} = \textit{Revert Features}$	$p_{25} = \textit{Maintenance Starts}$
$p_{26} = \textit{Maintenance Complete}$	

- Finite set of transitions

$$T = \{t_1, t_2, \dots, t_{26}\}$$

where

$t_0 = \text{Development}$	$t_1 = \text{Local Testing}$
$t_2 = \text{Local Testing Complete}$	$t_3 = \text{Attempting to commit \& run Pre-commit hooks}$
$t_4 = \text{Committing Changes}$	$t_5 = \text{Pushing Changes to Branch}$
$t_6 = \text{CI/CD Pipeline Starts}$	$t_7 = \text{CI/CD pass}$
$t_8 = \text{Unit Testing Pass}$	$t_{8f} = \text{Unit Testing Fail}$
$t_9 = \text{Integration Testing Pass}$	$t_{9f} = \text{Integration Testing Fail}$
$t_{10} = \text{System Testing Pass}$	$t_{10f} = \text{System Testing Fail}$
$t_{11} = \text{Deployment to Staging Environment starts}$	$t_{12} = \text{Deployment to Staging Environment done}$
$t_{13} = \text{Acceptance Testing Pass}$	$t_{13f} = \text{Acceptance Testing Fail}$
$t_{14} = \text{Staging development rolled back}$	$t_{15} = \text{Abandoning Feature}$
$t_{16} = \text{Feature Abandoned}$	$t_{17} = \text{Merging to Main}$
$t_{18} = \text{Tagging and Releasing}$	$t_{19} = \text{Deploying to Production}$
$t_{20} = \text{Deployment Monitoring Starts}$	$t_{21} = \text{Deployment Monitoring Passed}$
$t_{21f} = \text{Deployment Monitoring Failed}$	$t_{22} = \text{Production rolling back}$
$t_{23} = \text{Reverting Features Start}$	$t_{24} = \text{Reverting Features done}$
$t_{26} = \text{Maintenance}$	

- Flow relation

$$F = \left\{ \begin{array}{l} (p_0, t_0), (t_0, p_1), (p_1, t_1), (t_1, p_2), \\ (p_2, t_2), (t_2, p_1), (p_1, t_3), (t_3, p_3), \\ (p_3, t_4), (t_4, p_4), (p_4, t_5), (t_5, p_5), \\ (p_5, t_6), (t_6, p_6), (p_6, t_7), (t_7, p_7), \\ (p_7, t_{8f}), (t_{8f}, p_0), (p_7, t_8), (t_8, p_8), \\ (p_8, t_{9f}), (t_{9f}, p_0), (p_8, t_9), (t_9, p_9), \\ (p_9, t_{10f}), (t_{10f}, p_0), (p_9, t_{10}), (t_{10}, p_{10}), \\ (p_{10}, t_{11}), (t_{11}, p_{11}), (p_{11}, t_{12}), (t_{12}, p_{12}), \\ (p_{12}, t_{13f}), (t_{13f}, p_{13}), (p_{13}, t_{14}), (t_{14}, p_0), \\ (p_{13}, t_{15}), (t_{15}, p_{15}), (p_{15}, t_{16}), (t_{16}, p_0), \\ (p_{12}, t_{13}), (t_{13}, p_{17}), (p_{17}, t_{17}), (t_{17}, p_{18}), \\ (p_{18}, t_{18}), (t_{18}, p_{19}), (p_{19}, t_{19}), (t_{19}, p_{20}), \\ (p_{20}, t_{20}), (t_{20}, p_{21}), (p_{21}, t_{21f}), (t_{21f}, p_{22}), \\ (p_{22}, t_{22}), (t_{22}, p_{23}), (p_{23}, t_{23}), (t_{23}, p_{24}), \\ (p_{24}, t_{24}), (t_{24}, p_0), (p_{21}, t_{21}), (t_{21}, p_{25}), \\ (p_{25}, t_{26}), (t_{26}, p_{26}) \end{array} \right\}$$

- The initial marking M_0

$$M_0(p_1) = 1, \quad M_0(p_i) = 0 \quad \forall i \in \{1, 2, \dots, 26\}$$

4.1.1 Petri Net Graph

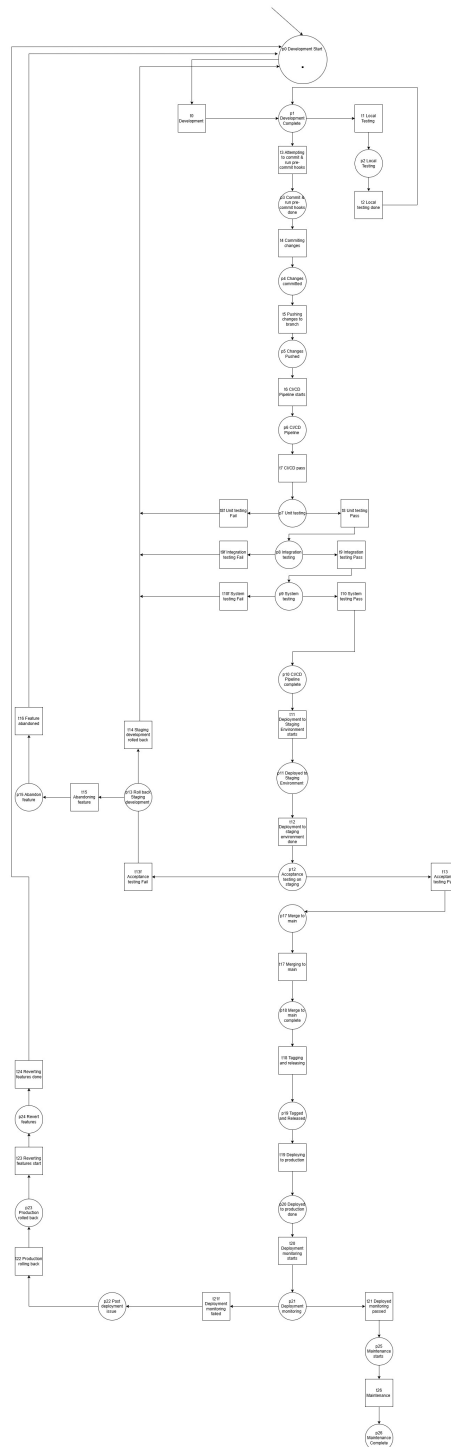


Figure 4.1: Petri net of the full software system, start to finish for one feature

4.1.1.1 Description

The places in the Petri net were chosen to represent the states of a software change as it progresses through the DevOps CI/CD pipeline, where each place corresponds to a condition that persists until an event occurs, which is represented as a transition. This is the case for the test places. For example, assume p_9 , i.e., System Testing holds the token while the respective tests are being conducted where it can only choose between the corresponding pass or fail transitions when the outcome of the test is determined.

The places in our Petri net represent how a singular feature progresses through our software system. Initially, the developer develops the feature and then commits and pushes it to a git branch, which is a simple and linear process. It then goes through a CI/CD pipeline created by DevOps, which triggers automatically on a git push and starts several tests with conditional outcomes. Here, we use XOR-splits to model the exclusivity of a test either passing or failing; for example p_7 (*Unit testing*) can either pass or fail but never both. Failing a test stage brings us back to to p_0 (*Development start*) as the developer has to then fix the issue. If all tests pass, the CI/CD pipeline is complete.

The feature is then staged, and acceptance testing is done. This again uses an XOR-split to model pass/fail. If it fails, additional work has to now be done to undo staging development. Here, it's possible the developer will decide to abandon the feature; if it's gotten this far through development but staging isn't acting as expected then the feature may be out of scope of the project, too complicated, or just need reworking in some fundamental way. If the feature is abandoned then we also return to p_0 (*Development start*), but this time with a new feature - it is difficult to model this change in perspective with a Petri net.

If acceptance testing passes then the feature is merged to the main branch and automatically tagged, released, and deployed. None of these steps are expected to fail. We then enter p_{21} (*Deployment monitoring*) which is a place that the system remains in for a long time, potentially months. After an amount of time, dependent on the feature, the feature is considered a success and monitoring passes, and the feature enters maintenance mode. If deployment monitoring reveals an issue then we take transition t_{21f} (*Deployment monitoring failed*), completely roll production back, revert the feature, and restart the system in order to fix the feature.

At any time there is only one token in the system, never more or less. This is because the token represents the feature, and the feature necessarily exists within the system for its entire duration.

4.1.2 Reachability Graph

4.1.3 Property Analysis

Here we analyse the reachability graph of our Petri net.

Boundedness. The Petri net is 1-bounded. The initial marking, p_0 , contains one token and no other initial markings contain any tokens. Tokens aren't generated with splits, and no more than one token is consumed at any point. So, every transition consumes and produces just one token and we have one token throughout.

Safety. The Petri net is safe, as a consequence of being 1-bounded.

Terminating. The Petri net is terminating. In the reachability graph there's a clear terminal node, coloured red. As there are no outgoing edges, if a token reaches here then the system

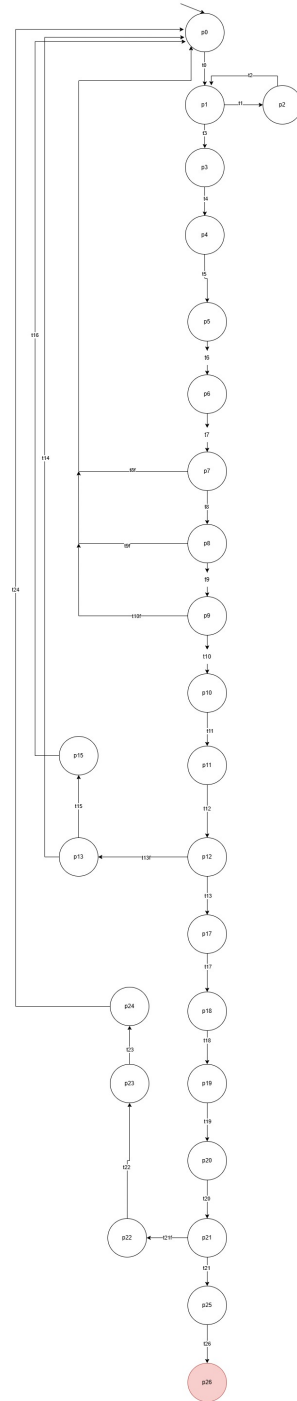


Figure 4.2: Reachability graph for 4.1, with terminal node in red

terminates. The various cycles in our reachability graph are irrelevant; they can prolong the time spent in the system but the terminal node is the logical end to our system and will be reached eventually.

Live. The Petri net is not live. Liveness and terminating properties are mutually exclusive, and our Petri net terminates.

Deadlock Free. The Petri net is not deadlock free, as the terminal node has no outgoing edges and the one token in our system can get stuck there, with transitions unable to fire.

Dead Transitions. The Petri net contains no dead transitions. There are no situations in which a transition would never have a token available to cause it to fire, as our Petri net is safe and all transitions are logical for the system - a successful workflow takes us down the simpler, linear path and the failure workflow allows us access to all cycles. The reachability graph demonstrates this, as when creating it we found no situation in which an edge didn't have two nodes.

Home Markings. The Petri net contains no home markings. A home marking a marking that can always be reached from any other marking, and the presence of our terminal marking p_{26} prevents any other marking from always being reachable.

Reversible. The Petri net is not reversible. Reversibility would require that our initial marking is a home marking, but we've established that our Petri net has no home markings. This is visible in the reachability graph; there would be a system-wide cycle if our graph was reversible.

4.2 Bug Fix Petri Net

Our Petri net system comprises the quadruple (P, T, F, m_0) :

- Finite set of places

$$P = \{p_1, p_2, \dots, p_{20}\}$$

where

$p_1 = \text{Bug reported}$	$p_2 = \text{Triage active}$
$p_3 = \text{Awaiting priority}$	$p_4 = \text{Normal classified}$
$p_5 = \text{Critical classified}$	$p_6 = \text{Normal queue}$
$p_7 = \text{Critical queue}$	$p_8 = \text{Normal queue complement}$
$p_9 = \text{Critical queue complement}$	$p_{10} = \text{Developer free}$
$p_{11} = \text{Developer assigned}$	$p_{12} = \text{Work in progress}$
$p_{13} = \text{Fix ready}$	$p_{14} = \text{CI running}$
$p_{15} = \text{Automated tests running}$	$p_{16} = \text{Tests passed}$
$p_{17} = \text{Security scan running}$	$p_{18} = \text{In review}$
$p_{19} = \text{Manual testing active}$	$p_{20} = \text{Deployed}$

- Finite set of transitions

$$T = \{t_1, t_2, \dots, t_{25}\}$$

where

$t_1 = \textit{Begin triage}$	$t_2 = \textit{Assess priority}$
$t_3 = \textit{Classify normal}$	$t_4 = \textit{Classify critical}$
$t_5 = \textit{Assign critical}$	$t_6 = \textit{Queue critical}$
$t_7 = \textit{Assign normal}$	$t_8 = \textit{Yield to critical}$
$t_9 = \textit{Queue normal}$	$t_{10} = \textit{Dequeue critical}$
$t_{11} = \textit{Dequeue normal}$	$t_{12} = \textit{Start work}$
$t_{13} = \textit{Submit fix}$	$t_{14} = \textit{Run CI}$
$t_{15} = \textit{Run automated tests}$	$t_{16} = \textit{Tests pass}$
$t_{17} = \textit{Tests fail}$	$t_{18} = \textit{Run security scan}$
$t_{19} = \textit{Scan passes}$	$t_{20} = \textit{Vulnerability found}$
$t_{21} = \textit{Review passes}$	$t_{22} = \textit{Review rejected}$
$t_{23} = \textit{Manual tests pass}$	$t_{24} = \textit{Manual tests fail}$
$t_{25} = \textit{Developer done}$	

- Flow relation

$$F = \left\{ \begin{array}{l} (p_1, t_1), (t_1, p_2), (p_2, t_2), (t_2, p_3), \\ (p_3, t_3), (t_3, p_4), (p_4, t_4), (t_4, p_5), \\ (p_5, t_5), (t_5, p_{10}), (p_{10}, t_5), (t_5, p_{11}), \\ (p_5, t_6), (t_6, p_9), (p_9, t_6), (t_6, p_7), \\ (p_4, t_7), (t_7, p_{10}), (p_{10}, t_7), (t_7, p_9), (t_7, p_{11}), \\ (p_4, t_8), (t_8, p_7), (p_7, t_8), (t_8, p_{10}), (t_8, p_6), (t_8, p_9), (t_8, p_{11}), \\ (p_4, t_9), (t_9, p_8), (p_8, t_9), (t_9, p_6), \\ (p_{10}, t_{10}), (t_{10}, p_7), (p_7, t_{10}), (t_{10}, p_9), (t_{10}, p_{11}), \\ (p_{10}, t_{11}), (t_{11}, p_6), (p_6, t_{11}), (t_{11}, p_9), (t_{11}, p_8), (t_{11}, p_{11}), \\ (p_{11}, t_{12}), (t_{12}, p_{12}), \\ (p_{12}, t_{13}), (t_{13}, p_{13}), (p_{13}, t_{14}), (t_{14}, p_{14}), \\ (p_{14}, t_{15}), (t_{15}, p_{15}), (p_{15}, t_{16}), (t_{16}, p_{16}), \\ (p_{15}, t_{17}), (t_{17}, p_{12}), (p_{16}, t_{18}), (t_{18}, p_{17}), \\ (p_{17}, t_{19}), (t_{19}, p_{18}), (p_{17}, t_{20}), (t_{20}, p_{12}), \\ (p_{18}, t_{21}), (t_{21}, p_{19}), (p_{18}, t_{22}), (t_{22}, p_{12}), \\ (p_{19}, t_{23}), (t_{23}, p_{20}), (p_{19}, t_{24}), (t_{24}, p_{12}), \\ (p_{20}, t_{25}), (t_{25}, p_{10}) \end{array} \right\}$$

- The initial marking M_0

$$M_0(p_1) = 1, \quad M_0(p_8) = 5, \quad M_0(p_9) = 1, \quad M_0(p_{10}) = 1, \quad M_0(p_i) = 0 \quad \forall i \in \{2, 3, 4, 5, 6, 7, 11, \dots, 20\}$$

4.2.1 Petri Net Graph

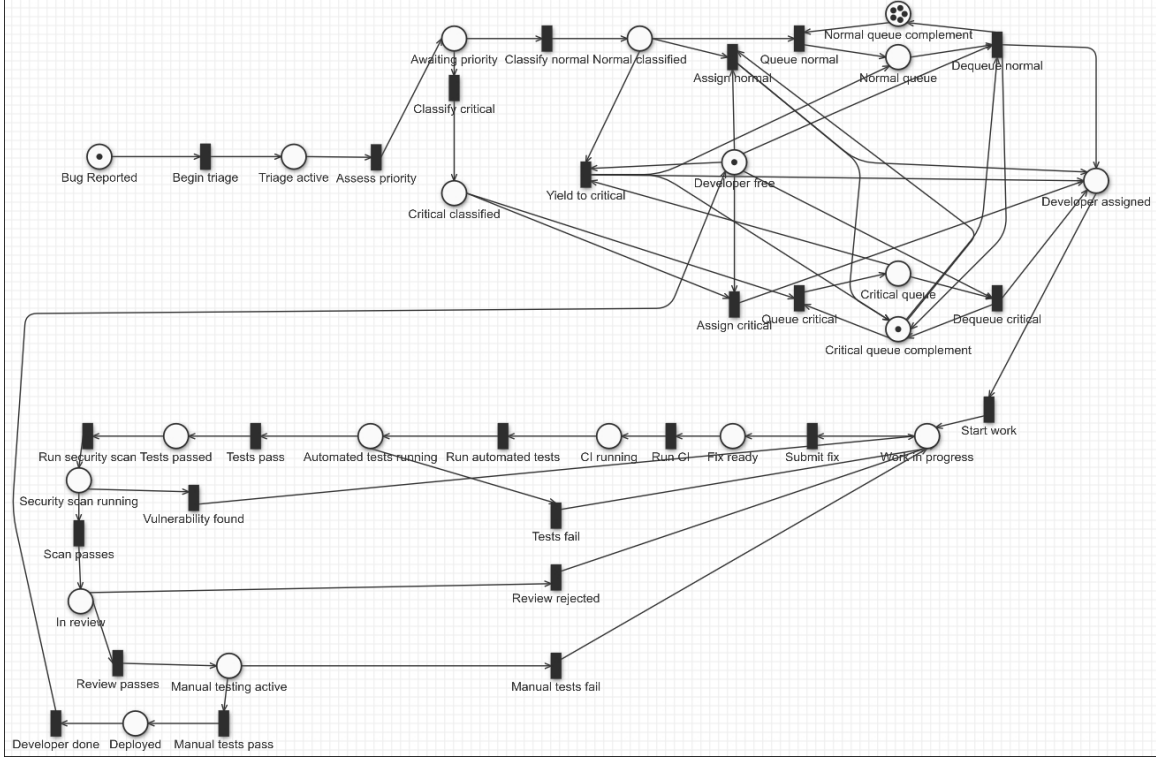


Figure 4.3: Petri net for a bug fix, with minimal labels for readability (see 4.2). *Bug Reported* is our initial place, with one token.

4.2.1.1 Description

Places were chosen conventionally; states between events. For example, p_8 (*Work in progress*) persists for as long as a developer is working on a bug fix and only enables t_{13} when the developer is done. The one non-obvious addition, p_{16} (*Tests passed*) separates security scanning from automated testing. Without it, both t_{16} (*Tests pass*) and t_{18} (*Run security scan*) would share an edge from p_{15} , creating a conflict and implying that the two are mutually exclusive instead of sequential. This is a modification from 3.2 where only a labeled edge is used to monitor this.

In our diagram there are many conditional outcomes, which are modeled as conflicts; two transitions both consume from the same place and only one of those transitions can fire. This shows mutual exclusivity at these points, and is applied to the priority branch (t_3 and t_4) as well as every pass/failure decision.

The loops caused by some failure don't return to our initial state p_1 , despite it being our only initial marking - instead they return to p_{12} . This models the fact that the entire process needn't start completely from scratch when a test is failed, and instead the developer can build on their existing, albeit flawed, bug fix. This also ensures that we don't skip any testing; any failure triggers a complete re-attempt.

The queueing system is modeled using four places: p_6 (*Normal queue*) and p_7 (*Critical queue*) old tokens representing other bugs waiting for a developer, while p_8 (*Normal queue complement*) and

p_9 (*Critical queue complement*) are their respective complements, starting at 5 and 1 respectively and therefore modeling a queue capacity of 5 and 1. This is the capacity pattern, and models the idea of our queues filling up and emptying.

When a critical bug is classified a developer is free, t_5 fires directly, bypassing any queuing. When a normal bug is classified but a critical bug is already waiting, then t_8 (*Yield to critical*) fires instead; the normal bug joins the normal queue while the critical bug is dequeued and assigned. p_9 (*Critical queue complement*) acts as a test for if the critical queue is empty, with both t_7 (*Assign normal*) and t_{11} (*Dequeue normal*) consuming tokens from it, meaning they're only enabled when the critical queue is empty.

The developer availability is modeled with p_{10} (*Developer free*), which holds a token when a developer is available. The token is consumed when any assignment or dequeue transitions fires and is only returned when t_{25} (*Developer done*) completes. This token ensures at most one bug is being worked on at a time. Bugs arriving when there's no free developer are held in the queue until the developer is free.

Our initial marking M_0 places a single token at p_1 , one token at p_{10} (developer starts free), one token at p_9 (critical queue starts empty), and five tokens at p_8 (normal queue starts empty). This models the start of a the lifecycle of a single bug entering an otherwise idle system.

4.2.2 Reachability Graph

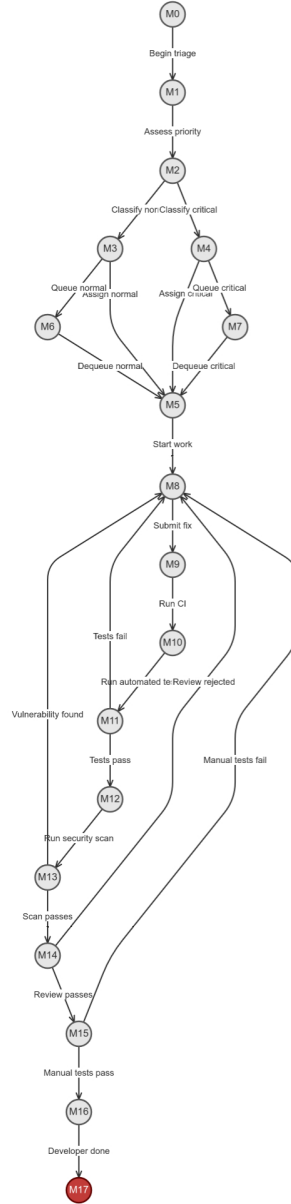


Figure 4.4: Reachability graph for bug fix Petri net, with minimal labels for readability (see 4.2), and terminal node in red. M_0 is the initial marking.

4.2.3 Property Analysis

Here we analyse the different properties of our bug fix Petri net. There are several important properties:

Boundedness. The net is bounded, we have a finite amount of tokens at each reachable marking. Because and every transition consumes exactly one token and produces exactly one token, the number of tokens is preserved throughout the Petri net regardless of the path taken, and we don't have any initial markings with unbounded tokens.

Safety. The net is not safe. While it is bounded, it is not 1-bounded. p_8 (*Normal queue complement*) starts with 5 tokens.

Terminating. The net is terminating, as there exists a reachable marking from which no transitions are enabled. t_{23} causes an all-zero marking after it fires and consumes the token in p_1 . This is the terminal node in the reachability graph, coloured red. Every path leads to this node, so the process always terminates.

Live. The net is not live. Liveness would mean there's no terminal node, however as seen in the reachability graph, p_{10} (*Developer free*) is indeed a terminal node once all bugs are fixed.

Deadlock Free. The net is not deadlock free, as it contains a terminal node p_{10} and can therefore deadlock there. This occurs after t_{25} fires. This corresponds to the terminal node in the reachability graph having no outgoing edges, as with liveness.

Dead Transitions. The net has no dead transitions. Every transition is reachable from M_0 along at least one path of execution, although t_3 and t_4 are mutually exclusive and only one can fire in any given bug fix, because they both consume from p_3 . These are not dead transitions, however, as they can be reached on different paths of the reachability graph.

Home Markings. There are no home markings. A home marking requires that from every reachable marking there exists a path back to it. Once again, our terminal marking M_{17} means that no transitions can fire and therefore no other markings are reachable from it. This means that no markings in our reachability graph can be reached from all markings.

Reversible. The net is not reversible. Reversibility means that M_0 should be reachable from every reachable marking, which makes it a home marking. However, no home markings exist. Additionally, our terminal marking M_{17} is a sink and therefore we can't reverse out of it.

Reflection

Overall, the models we created are effective in capturing the software process that we described.

Our state transition diagrams display a broad range of possible tests (from local, to automated, to acceptance on staging), and it enforces that all tests must pass before something is merged which is a logical and realistic process. It essentially models a modern git-based development process, including tagging, reverting, and commit hooks. Additionally, we model for features to be abandoned either at any point of progress or after a failed deployment. Each state diagram has a clear single feature as the subject, with a logical start and finish.

The downsides of our state transition diagrams is that all of our CI/CD stages run linearly, when this is typically done in parallel in practice, however this is an implementation concern and not particularly possible in a state transition diagram. In order to fix this we would have had to collapse every test into a single state, which loses too much detail to be an acceptable trade-off. We also don't mention any actors, instead choosing a more neutral perspective of a feature or bug fix. We also assume that every bug report is accurate, with no notion of dismissable during triage, and due to the limitations of state transition diagrams we cannot explicitly show that critical and normal queues have different priorities, only implicitly.

Our Petri nets utilise the features that are available over state transition diagrams to model more complicated ideas; the bug fix Petri net having queues with differing capacities and developer availability, and showing that critical bugs take priority over normal ones. The nets also have a clear flow from reported to deployed, and in general match and expand on our state transition systems.

On the other hand, our Petri nets only model a single developer that can fix a single bug at once. Changing this would have added vastly more complexity to our project, and it still models the software system that a smaller company such as a startup may use, but is worth mentioning regardless. Additionally, there is a logical flaw in our bug fix Petri net where "Assign critical" and "Assign normal" both produce output tokens in "Developer assigned", skipping the queues we've set up instead of deferring to them first.

The first Petri net could have shown test parallelism through splits and joins, but the diagram is meant to gate progress on required steps, not to prescribe how those steps are executed. We could have done this by having the test be at the same level and to be done at the same time rather than doing it linearly. The advantages would be that it's more accurate to real-life practices, but modeling this is complicated and too convoluted in an already large Petri net.

One area in which our first Petri net is weak is the deployment process; we have a place for *deployment stage testing* leading into *post-deployment monitoring testing*, which implies that deployment has taken place though the transition that connects them does not explicitly state as such. Moreover, there is some confusion between release and deployment, as they are treated as separate, sequential steps rather than a cohesive unit. We could have avoided this by collapsing this into a

single state *deployed* and moving the acceptance testing beforehand.

Our reachability graphs behave as we expect. In particular, the fact that they're bounded and terminating confirms that this system models a feature or fix from start to finish as one self-contained, completable action. With a more complicated software system, where features and bugs are continuously revisited, we could perhaps expect a live system, but by-and-large our work represents a real-world system.

The key takeaway we had is that there is no one-size-fits-all modeling technique. The state transition diagrams are useful for their simplicity and readability, while Petri nets allow the modeler to make the flow of control more explicit and introduce more complex conditions and concurrency using tokens, AND-splits, XOR-splits, and patterns like the capacity pattern.

The work was split into four major stages:

1. Writing the process description
2. Creating the two state transition diagrams
3. Creating the two Petri nets
4. Writing the reflection

Each team member's contributions are detailed in Table 5.1.

Section	Reference	Author
Process description	chapter 2	Conrad Clough
Transition diagram 1	subsection 3.1.1	Anton Ruban
Transition diagram 2	subsection 3.2.1	Torben Freise
Model descriptions (both transition diagrams)	subsubsection 3.1.1.1, subsubsection 3.2.1.1	Conrad Clough
Petri net 1	section 4.1	Anton Ruban
Petri net 2	section 4.2	Conrad Clough
Reflection	chapter 5	Torben Freise

Table 5.1: Authorship of each section.

Bibliography